

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import warnings
warnings.filterwarnings('ignore')

```

```
df = pd.read_csv('https://d2beiqkhq929f0.cloudfront.net/public_assets/assets/000/15')
```

```
df.head()
```

	Make	Model	Vehicle Class	Engine Size(L)	Cylinders	Transmission	Fuel Type	Fuel Consumption City (L/100 km)	Fuel Consumption Hwy (L/100 km)
0	ACURA	ILX	COMPACT	2.0	4	AS5	Z	9.9	6.7
1	ACURA	ILX	COMPACT	2.4	4	M6	Z	11.2	7.7
2	ACURA	ILX HYBRID	COMPACT	1.5	4	AV7	Z	6.0	5.8
3	ACURA	MDX 4WD	SUV - SMALL	3.5	6	AS6	Z	12.7	9.1
4	ACURA	RDX AWD	SUV - SMALL	3.5	6	AS6	Z	12.1	8.7

```
df.info()
```

```
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 7385 entries, 0 to 7384
```

```
Data columns (total 12 columns):
```

#	Column	Non-Null Count	Dtype
0	Make	7385 non-null	object
1	Model	7385 non-null	object
2	Vehicle Class	7385 non-null	object
3	Engine Size(L)	7385 non-null	float64
4	Cylinders	7385 non-null	int64
5	Transmission	7385 non-null	object
6	Fuel Type	7385 non-null	object
7	Fuel Consumption City (L/100 km)	7385 non-null	float64
8	Fuel Consumption Hwy (L/100 km)	7385 non-null	float64
9	Fuel Consumption Comb (L/100 km)	7385 non-null	float64
10	Fuel Consumption Comb (mpg)	7385 non-null	int64
11	CO2 Emissions(g/km)	7385 non-null	int64

```
dtypes: float64(4), int64(3), object(5)
```

```
memory usage: 692.5+ KB
```

	Engine Size(L)	Cylinders	Fuel Consumption City (L/100 km)	Fuel Consumption Hwy (L/100 km)	Fuel Consumption Comb (L/100 km)	Fuel Consumption Comb (mpg)	Emission
count	7385.000000	7385.000000	7385.000000	7385.000000	7385.000000	7385.000000	7385.00
mean	3.160068	5.615030	12.556534	9.041706	10.975071	27.481652	250.58
std	1.354170	1.828307	3.500274	2.224456	2.892506	7.231879	58.51
min	0.900000	3.000000	4.200000	4.000000	4.100000	11.000000	96.00
25%	2.000000	4.000000	10.100000	7.500000	8.900000	22.000000	208.00
50%	3.000000	6.000000	12.100000	8.700000	10.600000	27.000000	246.00
75%	3.700000	6.000000	14.600000	10.200000	12.600000	32.000000	288.00
max	8.400000	16.000000	30.600000	20.600000	26.100000	69.000000	522.00

quality check

```
# Check for missing values
print("Missing values per column:")
print(df.isnull().sum())

# Check and remove duplicates
duplicates = df.duplicated().sum()
print(f"\nDuplicate rows found: {duplicates}")

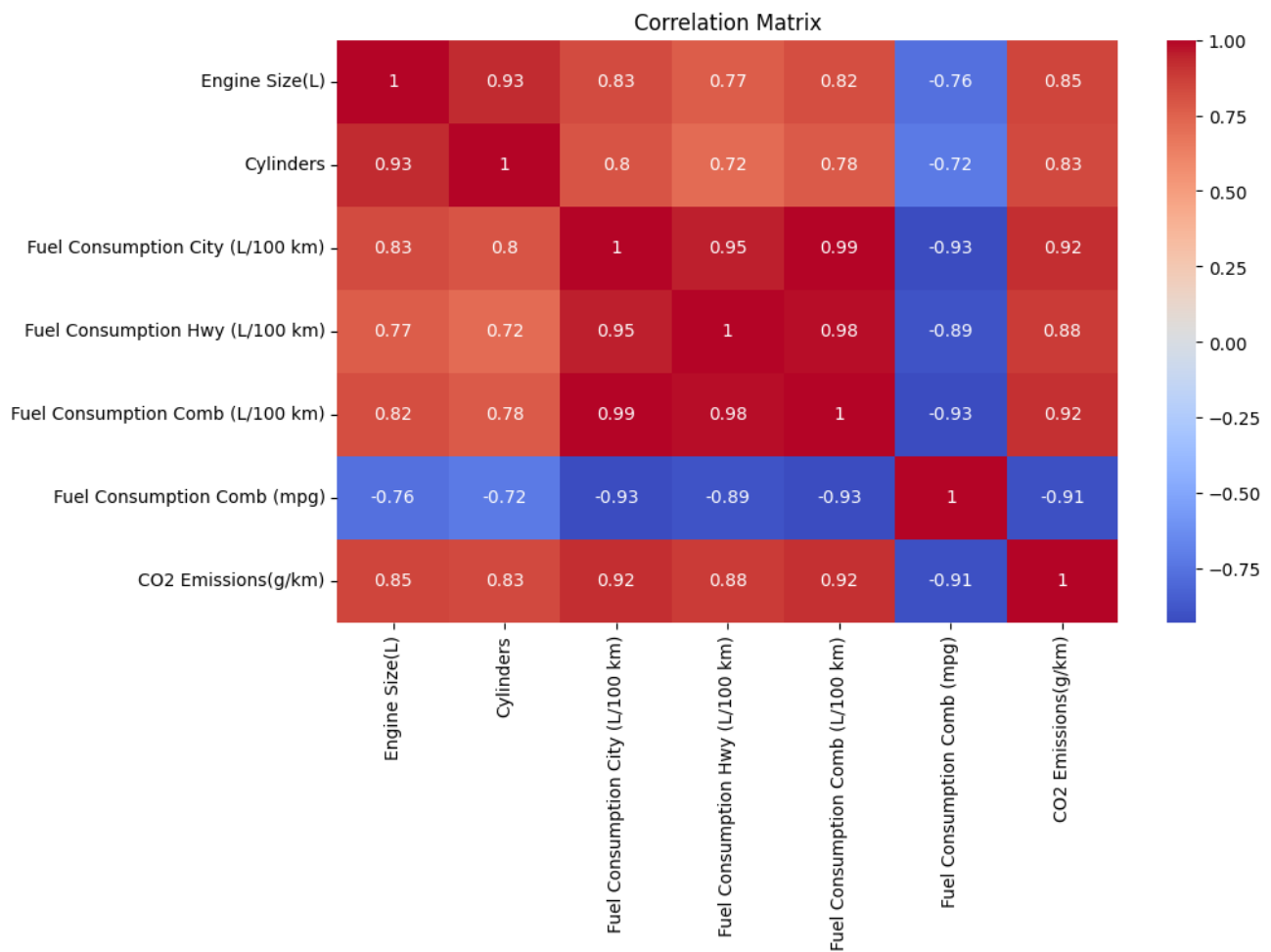
if duplicates > 0:
    df = df.drop_duplicates()
    print(f"Removed {duplicates} duplicate rows. New dataset shape: {df.shape}")
else:
    print("No duplicate rows found.")

Missing values per column:
Make                                0
Model                              0
Vehicle Class                       0
Engine Size(L)                     0
Cylinders                           0
Transmission                        0
Fuel Type                           0
Fuel Consumption City (L/100 km)    0
Fuel Consumption Hwy (L/100 km)     0
Fuel Consumption Comb (L/100 km)    0
Fuel Consumption Comb (mpg)         0
CO2 Emissions(g/km)                 0
dtype: int64

Duplicate rows found: 1103
Removed 1103 duplicate rows. New dataset shape: (6282, 12)

#correlation Analysis

plt.figure(figsize=(10,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```



Relationship between numeric column features and co2 emission

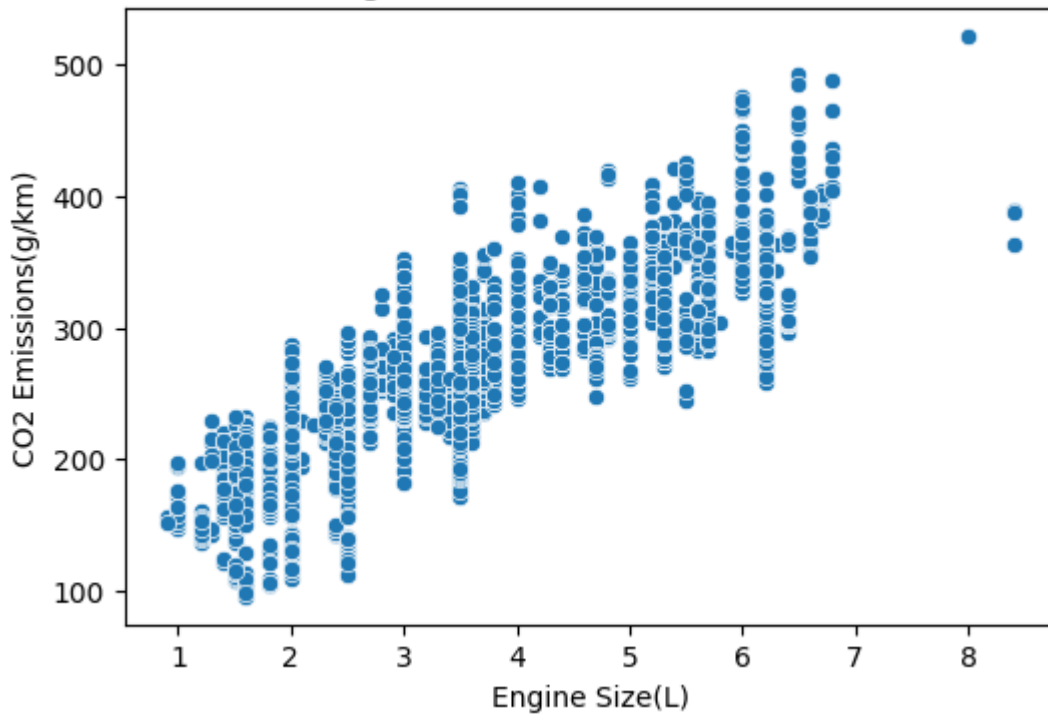
```
numeric_cols = df.select_dtypes(include=['number']).columns
print(numeric_cols)
```

```
Index(['Engine Size(L)', 'Cylinders', 'Fuel Consumption City (L/100 km)',
      'Fuel Consumption Hwy (L/100 km)', 'Fuel Consumption Comb (L/100 km)',
      'Fuel Consumption Comb (mpg)', 'CO2 Emissions(g/km)'],
      dtype='object')
```

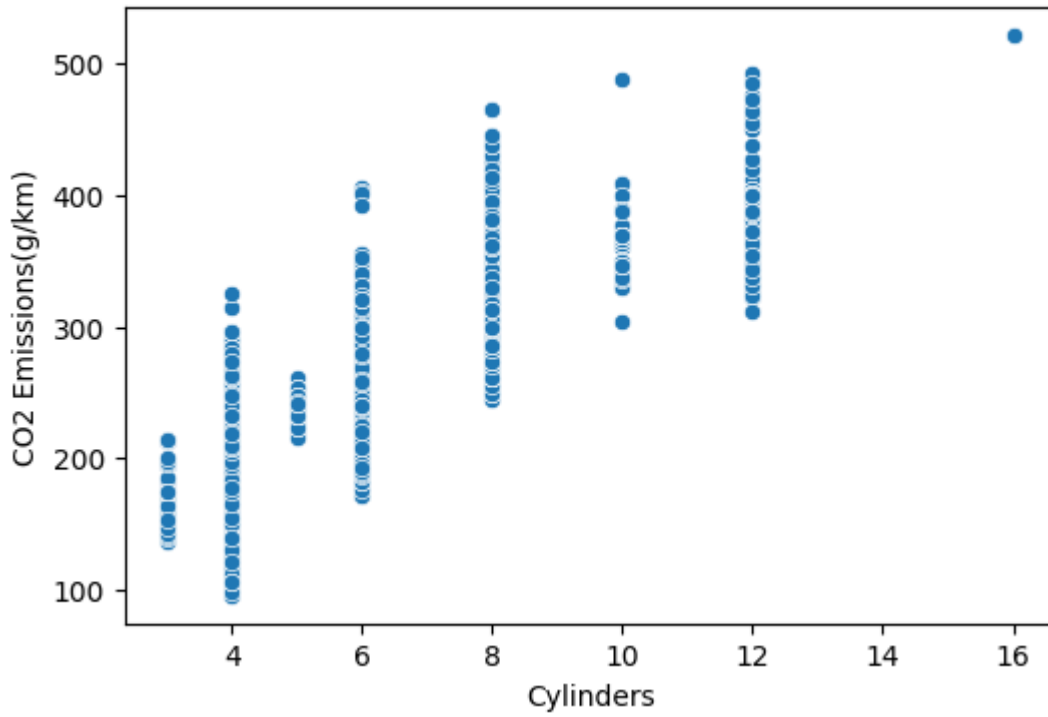
```
numeric_features = ['Engine Size(L)', 'Cylinders', 'Fuel Consumption City (L/100 km)',
                    'Fuel Consumption Hwy (L/100 km)', 'Fuel Consumption Comb (L/100 km)']
```

```
for col in numeric_features:
    plt.figure(figsize=(6,4))
    sns.scatterplot(x=df[col], y=df['CO2 Emissions(g/km)'])
    plt.title(f'{col} vs CO2 Emissions')
    plt.show()
```

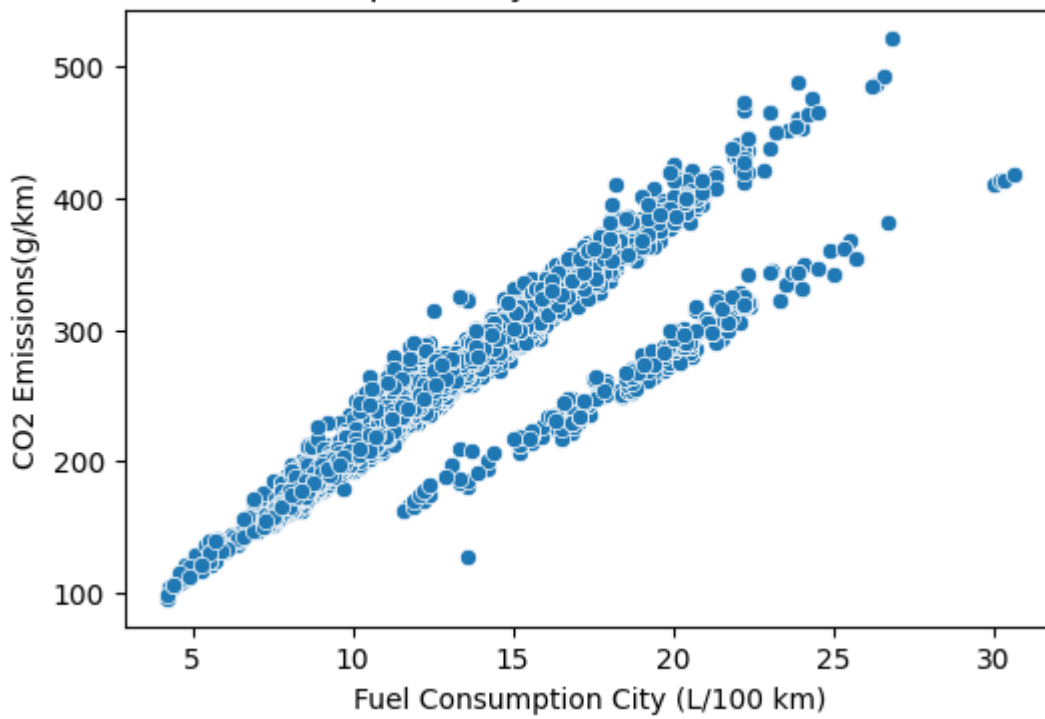
Engine Size(L) vs CO2 Emissions



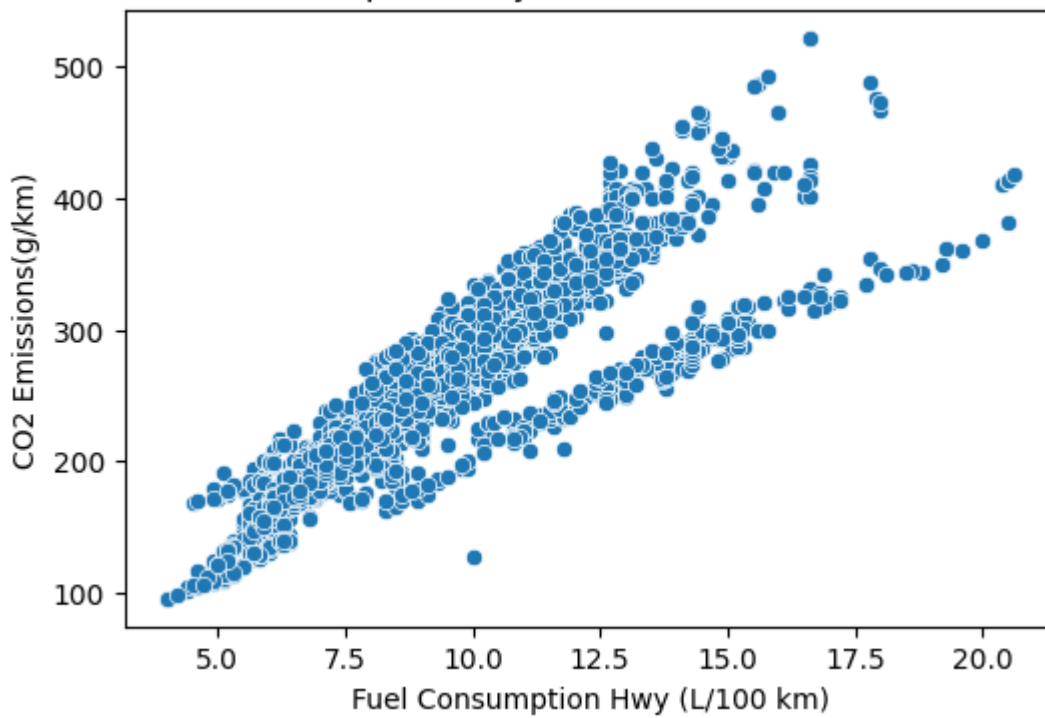
Cylinders vs CO2 Emissions

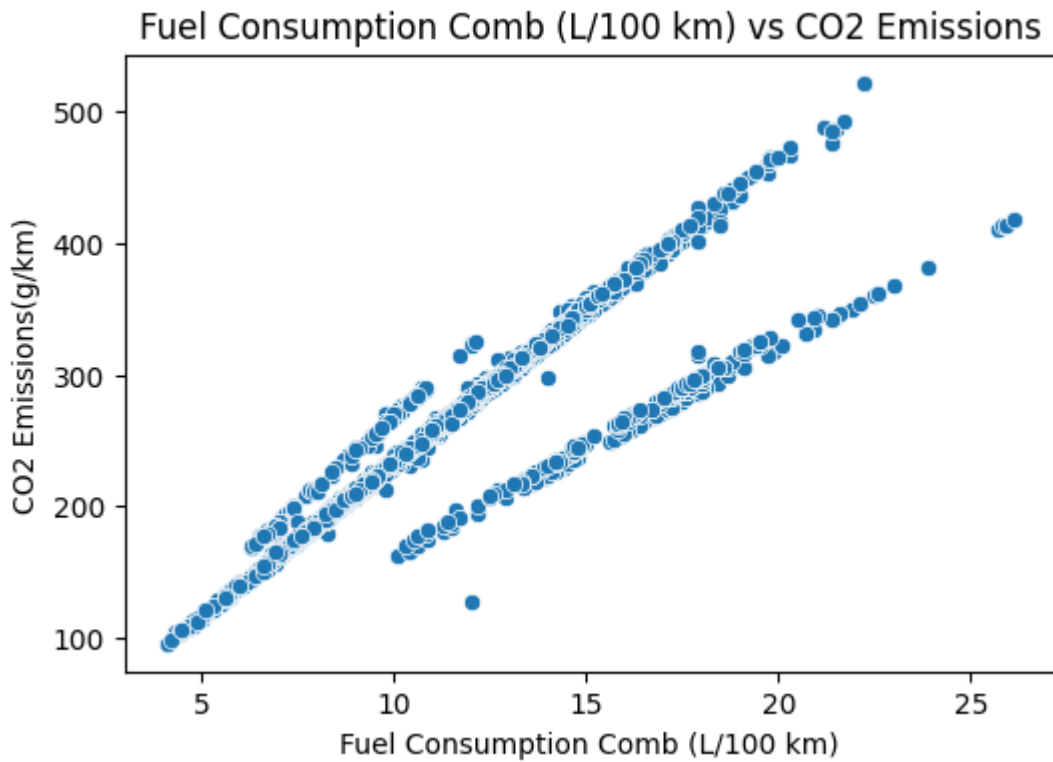


Fuel Consumption City (L/100 km) vs CO2 Emissions



Fuel Consumption Hwy (L/100 km) vs CO2 Emissions

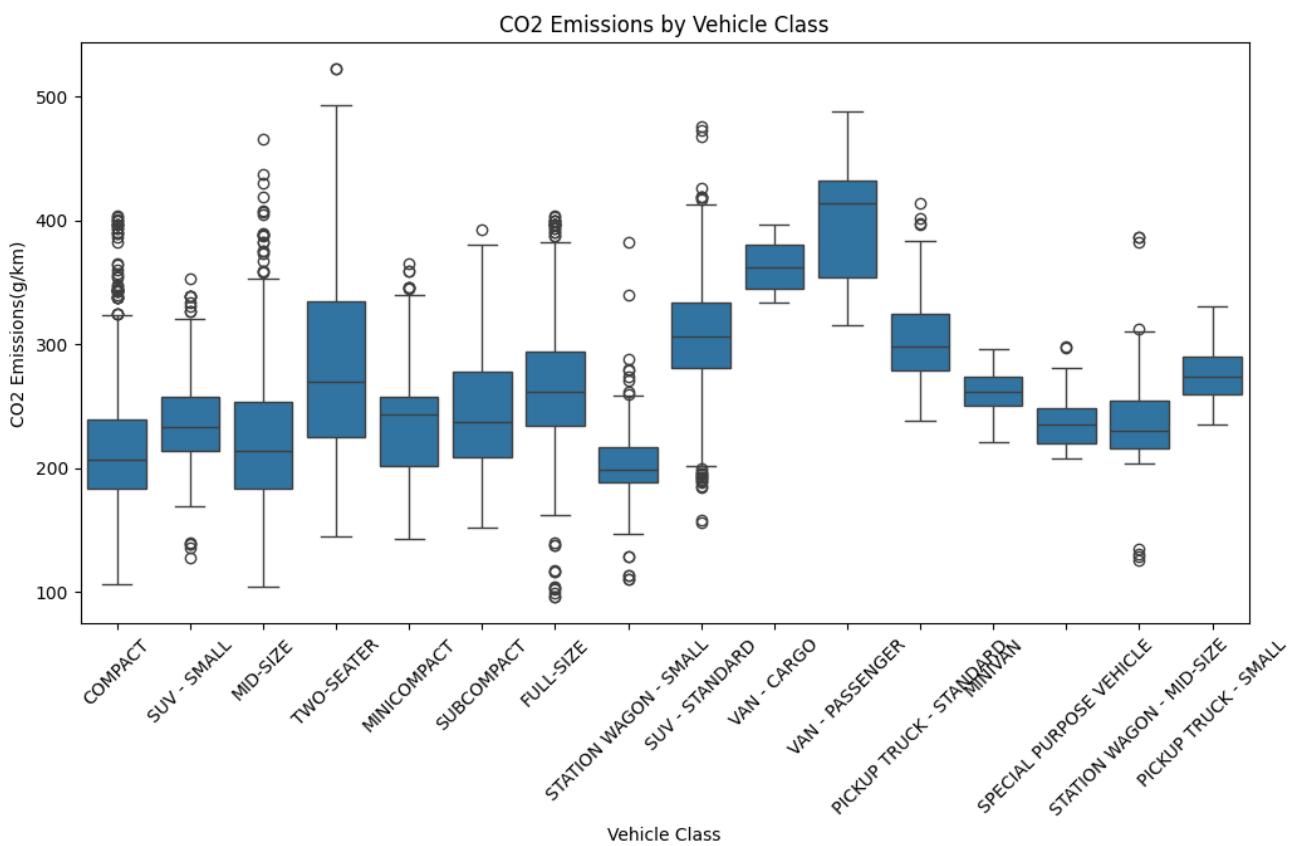
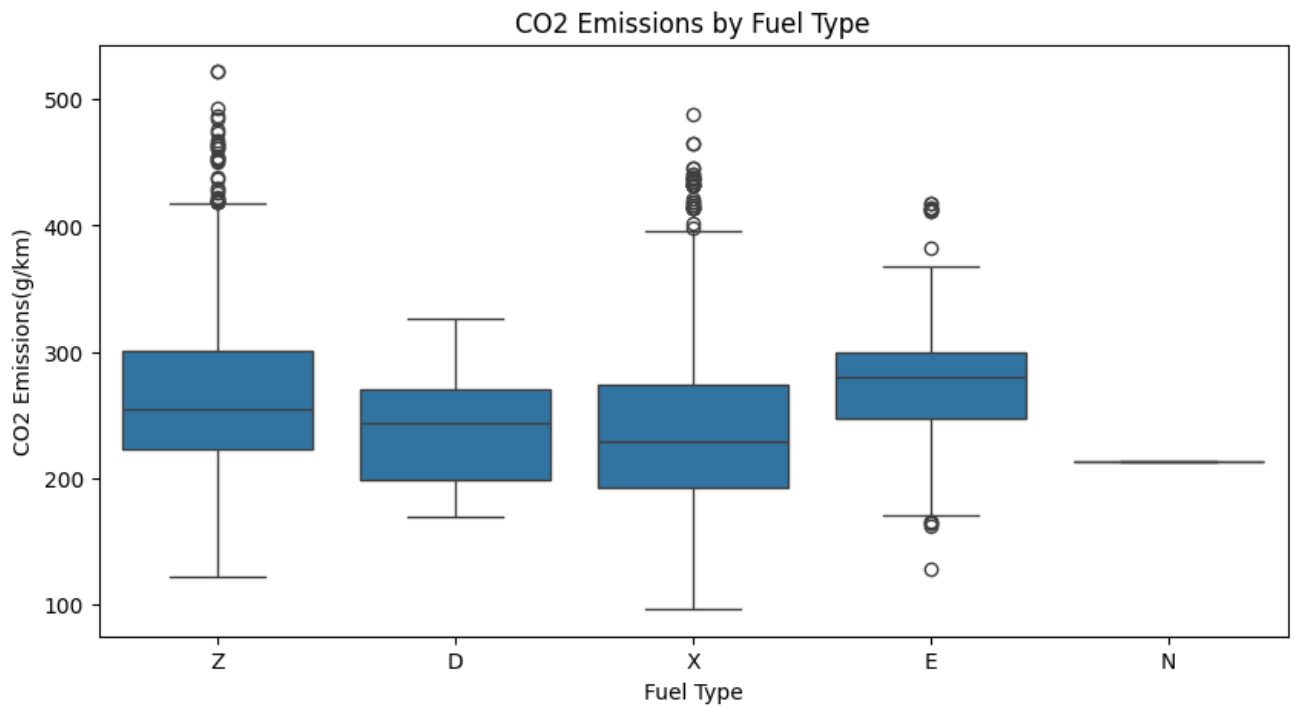




CO2 emissions by fuel type and vehicle class

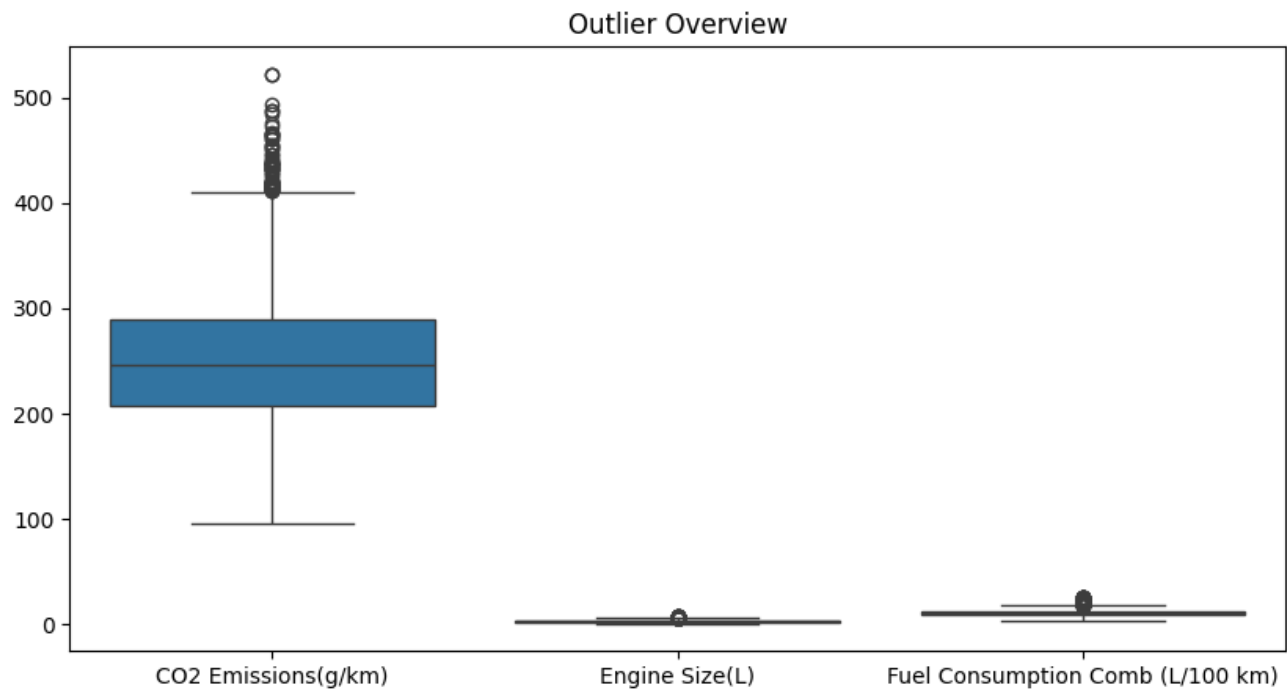
```
plt.figure(figsize=(10,5))
sns.boxplot(x='Fuel Type', y='CO2 Emissions(g/km)', data=df)
plt.title('CO2 Emissions by Fuel Type')
plt.show()

plt.figure(figsize=(12,6))
sns.boxplot(x='Vehicle Class', y='CO2 Emissions(g/km)', data=df)
plt.xticks(rotation=45)
plt.title('CO2 Emissions by Vehicle Class')
plt.show()
```



OUTLIER DETECTION

```
plt.figure(figsize=(10,5))
sns.boxplot(data=df[['CO2 Emissions(g/km)', 'Engine Size(L)', 'Fuel Consumption Cor
plt.title('Outlier Overview')
plt.show()
```



Prepare data for modeling

```
# Select numeric + encoded categorical features
df_model = df.copy()
df_model = pd.get_dummies(df_model, columns=['Transmission', 'Fuel Type', 'Vehicle Cl

X = df_model.drop(['CO2 Emissions(g/km)', 'Model'], axis=1)
y = df_model['CO2 Emissions(g/km)']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

#print(X_train)

#print(X_test)
```

liner regression model create

```
lr = LinearRegression()
lr.fit(X_train, y_train)

y_pred = lr.predict(X_test)

print(y_pred)

[308.41942625 244.42183495 296.07113144 ... 300.68148554 194.18557692
 194.18647174]
```

model validation

```

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"MAE: {mae:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R2 Score: {r2:.3f}")

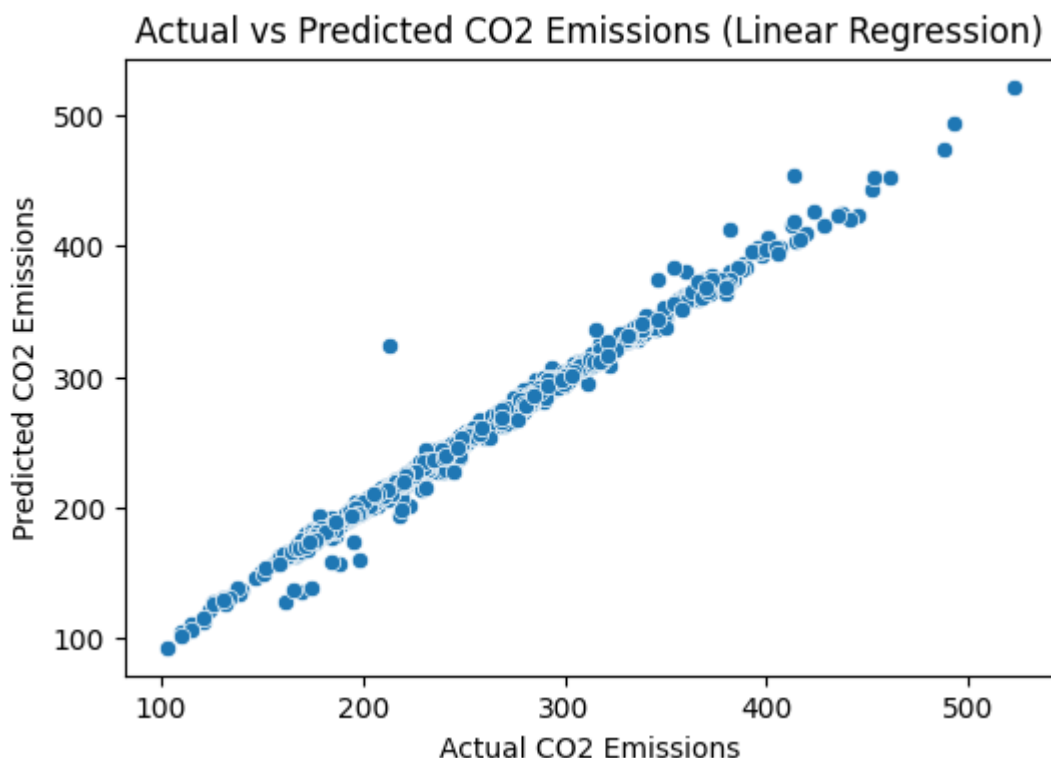
MAE: 3.24
RMSE: 6.03
R2 Score: 0.990

plt.figure(figsize=(6,4))
sns.scatterplot(x=y_test, y=y_pred)
plt.xlabel('Actual CO2 Emissions')
plt.ylabel('Predicted CO2 Emissions')
plt.title('Actual vs Predicted CO2 Emissions (Linear Regression)')
plt.show()

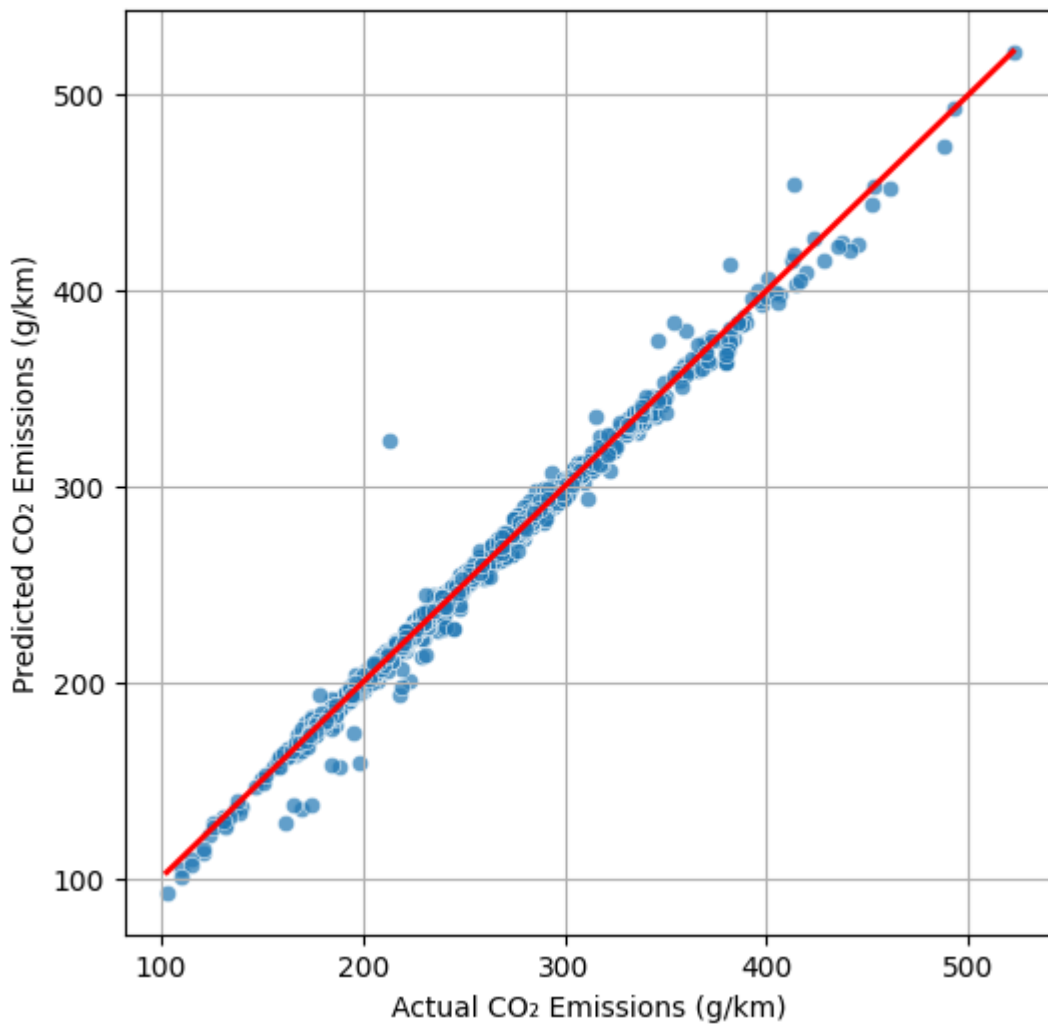
plt.figure(figsize=(6,6))
sns.scatterplot(x=y_test, y=y_pred, alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', l
plt.title('Actual vs Predicted CO2 Emissions')
plt.xlabel('Actual CO2 Emissions (g/km)')
plt.ylabel('Predicted CO2 Emissions (g/km)')
plt.grid(True)
plt.show()

plt.figure(figsize=(6,4))
sns.histplot(y_test - y_pred, kde=True)
plt.title('Residual Distribution')
plt.show()

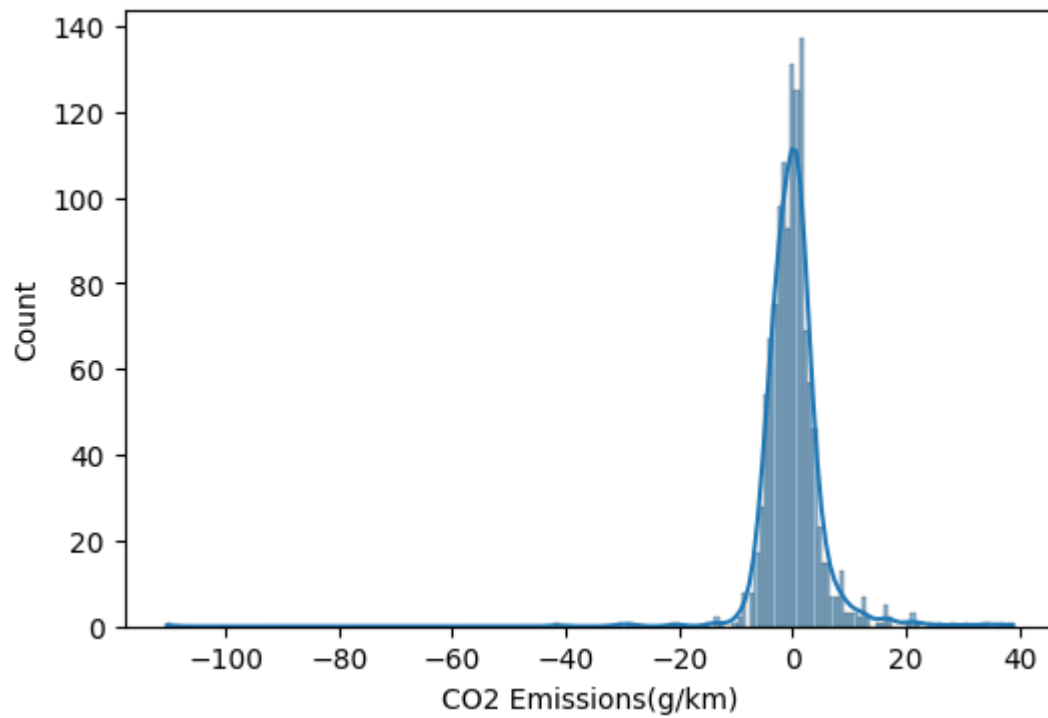
```



Actual vs Predicted CO₂ Emissions



Residual Distribution



model interpretation

```
coeffs = pd.DataFrame({'Feature': X_train.columns, 'Coefficient': lr.coef_})  
coeffs.sort_values(by='Coefficient', ascending=False).head(10)
```

	Feature	Coefficient
56	Make_BUGATTI	24.246047
72	Make_LAMBORGHINI	9.018598
60	Make_CHRYSLER	8.241485
54	Make_BENTLEY	7.831579
4	Fuel Consumption Comb (L/100 km)	7.407662
84	Make_ROLLS-ROYCE	7.360459
2	Fuel Consumption City (L/100 km)	6.550890
76	Make_MASERATI	6.326618
52	Make_ASTON MARTIN	5.955151
61	Make_DODGE	5.934205

Conclusion

- Engine size, number of cylinders, and combined fuel consumption show strong positive correlation with CO₂ emissions.
- Vehicles using **diesel** or **premium gasoline** tend to emit higher CO₂ compared to **regular fuel**.
- Linear Regression provides a simple and interpretable model to estimate emissions based on vehicle specifications.
- Future improvements can include regularization or feature selection to reduce overfitting.